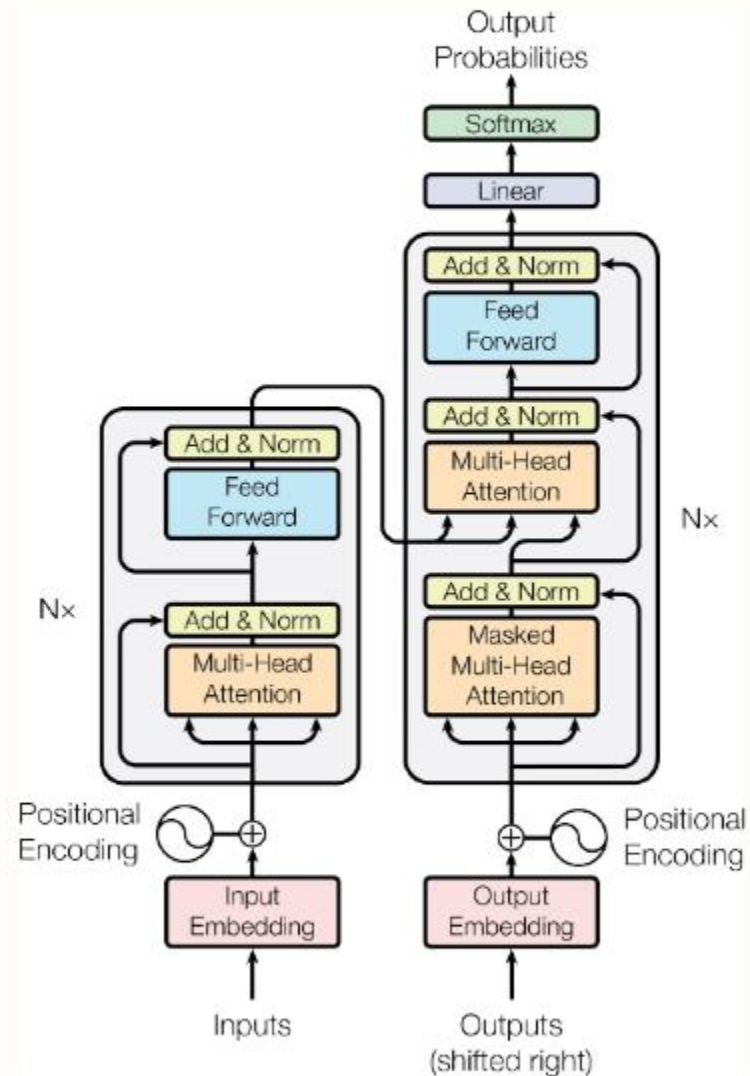


The Original Transformer Architecture

The 2017 paper assembled familiar components into an encoder-decoder system.

READ THE PUBLISHED DIAGRAM FROM BOTTOM TO TOP



What to recognize

- 1 Encode the source sequence**
The left stack alternates self-attention and a position-wise feedforward network.
- 2 Mask the target-side self-attention**
The decoder can use earlier outputs, but it cannot look at future target tokens.
- 3 Revisit the encoder through cross-attention**
The middle decoder attention block queries the source representation at every output step.
- 4 Produce next-token probabilities**
A linear vocabulary head and softmax turn the final decoder state into a distribution.

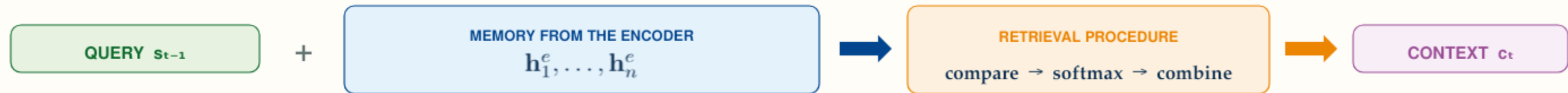
Decoder-only LLMs keep the masked decoder stack—but remove the encoder and cross-attention.

Cross-Attention and Self-Attention

The retrieval procedure stays the same. The source of the query and memory changes.

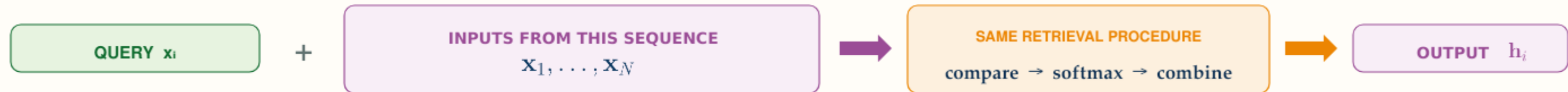
ATTENTION INPUTS

CROSS-ATTENTION



A decoder state retrieves source information from a different sequence.

SELF-ATTENTION



Each position retrieves information from allowed positions in the same sequence.

Causal attention sees the current and earlier positions.
A bidirectional encoder may attend to positions on both sides.

The Transformer makes the comparison and retrieval roles explicit through queries, keys, and values.

Database Lookup and Attention

A query identifies relevant entries. Keys support matching, and values contain the information to retrieve.

DATABASE LOOKUP

QUERY

book_id = 'BK205'

Find the title and shelf
for this book ID.



BOOK CATALOGUE

book_id (key)	Title (value)	Shelf (value)
BK101	Linear Algebra	A2
BK205	Language Models	B4
BK310	Optimization	C1



RETURNED RECORD

Language Models
Shelf B4

SOFT ATTENTION

QUERY VECTOR

q_i

Compare with each key.
Softmax gives weights.



Key vector	Value vector	Weight
k_1	v_1	0.10
k_2	v_2	0.80
k_3	v_3	0.10



WEIGHTED RESULT

$$0.10v_1 + 0.80v_2 + 0.10v_3$$

Values contribute in
proportion to their
weights.

Illustrative weights. Attention combines vector values, not catalogue text.

Soft retrieval combines several values. Differentiability lets training adjust the matching rule.

Role-Specific Representations

SGNS and attention use separate vectors for the two sides of a dot-product score.

RELATION TO SGNS

SGNS: STORED EMBEDDINGS

Word	Target vector	Context vector
doctor	$\mathbf{W}_{\text{doctor}}$	$\mathbf{C}_{\text{doctor}}$
medicine	$\mathbf{W}_{\text{medicine}}$	$\mathbf{C}_{\text{medicine}}$

The same word has a separate learned vector in each embedding table.

DOT-PRODUCT SCORE

$$\mathbf{W}_{\text{doctor}} \cdot \mathbf{C}_{\text{medicine}}$$

Target “doctor” with context “medicine”

SGNS learns word-type embeddings. Attention computes vectors from the current token representations.

SELF-ATTENTION: COMPUTED VECTORS

INPUT REPRESENTATION AT POSITION i
 $\mathbf{x}_i$

$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}_Q$$

$$\mathbf{k}_i = \mathbf{x}_i \mathbf{W}_K$$

Different learned matrices allow different vectors for the same token.

DOT-PRODUCT SCORE

$$\mathbf{q}_i \cdot \mathbf{k}_j$$

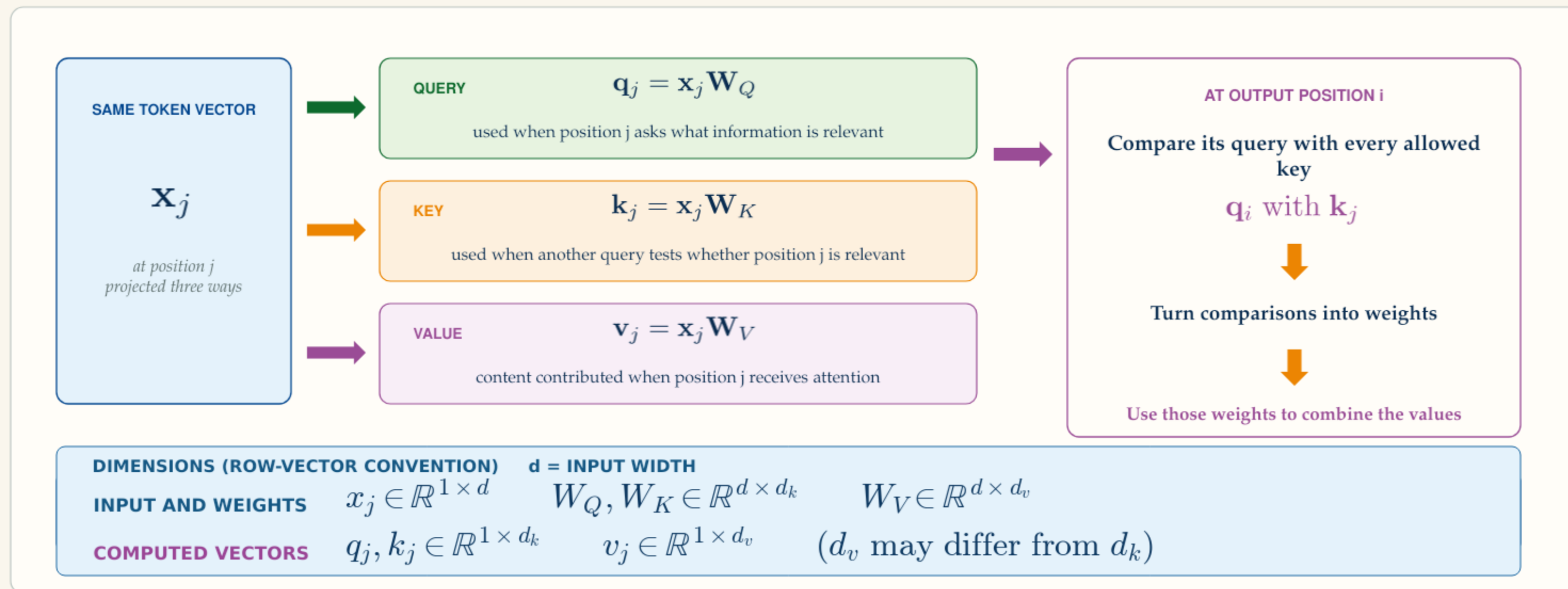
Query at position i with key at position j

Different roles motivate separate query and key projections. Values carry the content to retrieve.

Query, Key, and Value Projections

Each token representation is projected into three roles used by the attention procedure.

PROJECTION ROLES



We will apply these projections to the target sequence of a translation decoder.

Vaswani et al. (2017)

Hindi-to-English Translation Example

The encoder holds the Hindi source while the decoder generates the English target.

A SOURCE SENTENCE AND ONE TARGET STEP

HINDI SOURCE • ENCODER

बिल्ली सोई।

Available at every decoder step



ENGLISH TARGET • DECODER

The cat slept.

Generated one token at a time

DECODER STEP: PREDICT “slept”

<BOS>

The

cat



slept

DECODER SELF-ATTENTION

English prefix: <BOS>, The, cat

DECODER CROSS-ATTENTION

Complete Hindi source

The next slide expands this decoder example into four shifted target positions.

Shifted Decoder Targets

Each English target is predicted using the prefix available at that decoder position.

AVAILABLE PREFIXES AND NEXT-TOKEN TARGETS

Query row	⟨BOS⟩	The	cat	slept	Next token
Row 1	⟨BOS⟩	The	cat	slept	The
Row 2	⟨BOS⟩	The	cat	slept	cat
Row 3	⟨BOS⟩	The	cat	slept	slept
Row 4	⟨BOS⟩	The	cat	slept	.

Blue: earlier positions Purple: current query Red: future tokens unavailable to this query

Training provides the complete shifted input: ⟨BOS⟩, The, cat, slept.

Each row defines a supervised next-token prediction.

The decoder builds a representation for every query position.

Each prediction uses the complete Hindi source and its available English prefix.

Attention in Translation

The encoder-decoder Transformer uses attention in three settings.

ATTENTION ROLES

Hindi source: बिल्ली सोई।

English target: The cat slept.

Attention	Query source	Key and value source	Accessible positions	Purpose
Encoder self-attention	Source representations	Source representations	Entire source sentence	Build contextual source representations
Decoder self-attention	Target representations	Target representations	Current and earlier decoder inputs	Represent the available target prefix
Cross-attention	Decoder representations	Encoder outputs	Entire source sentence	Retrieve source information for translation

At “cat”, target self-attention can use ⟨BOS⟩, The, cat. Cross-attention can use the full Hindi source.

The decoder uses causal self-attention for target context and cross-attention for source context.

Parallel Computation in Attention

Attention can evaluate query positions together once their input representations are available.

COMPUTATIONAL DEPENDENCIES

CROSS-ATTENTION

Each decoder query retrieves information directly from encoder outputs.
Multiple queries can be evaluated together once they are available.

RNN RECURRENCE

Each state requires the preceding state.



Adding cross-attention leaves this recurrent dependency in place.

SELF-ATTENTION

Every query uses input rows already available to this layer.



Positions can be computed together within a layer during training.

Large matrix operations make effective use of GPUs.

Decoder training must preserve the information available at each prediction.

Causality During Training

The complete target input is available during training. Each prediction has a limited target prefix.

PREDICTION AT THE DECODER POSITION FOR “cat”

Hindi source: बिल्ली सोई।

English target: The cat slept.

Decoder position	1	2	3	4
Training input	⟨BOS⟩	The	cat	slept
Available to query 3	Permitted	Permitted	Permitted	Unavailable

The query at “cat” must predict “slept”.

Unrestricted attention could read “slept” from input position 4.

During generation, this query has received only ⟨BOS⟩, The, cat.

A causal mask excludes future positions before softmax.

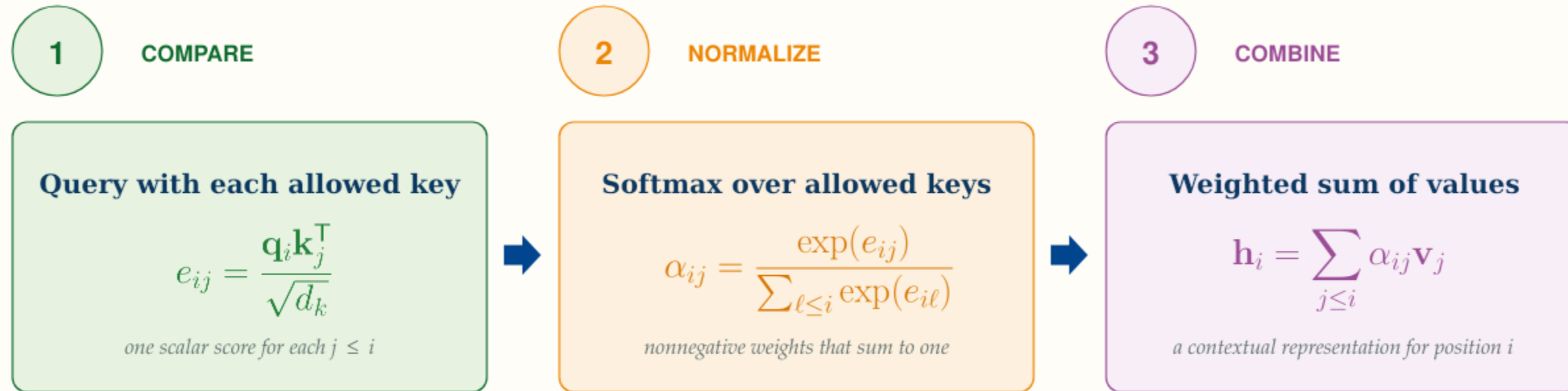
The current position is allowed because its prediction target is the next token.

The mask restricts information access while matrix operations evaluate positions together.

Decoder Causal Self-Attention

At target position i , the decoder can attend to target positions $j \leq i$.

ATTENTION AT A DECODER POSITION



WHY SCALE THE DOT PRODUCT?

As d_k grows, dot products can become large. Scaling controls the softmax input and helps preserve useful gradients during training.

The causal rule restricts each query to its available decoder prefix.

Vaswani et al. (2017)

Toy Decoder Representations

Use binary embeddings to examine the query at “cat”, whose training target is “slept”.

DECODER INPUT: ⟨BOS⟩, The, cat, slept

EMBEDDINGS, KEYS AND VALUES

Token	$\mathbf{x}_j$ (1×3)	$\mathbf{k}_j$ (1×2)	$\mathbf{v}_j$ (1×2)
⟨BOS⟩	$\begin{bmatrix} 0 & 0 & 0 \end{bmatrix}$	$\begin{bmatrix} 0 & 0 \end{bmatrix}$	$\begin{bmatrix} 0 & 0 \end{bmatrix}$
The	$\begin{bmatrix} 1 & 0 & 0 \end{bmatrix}$	$\begin{bmatrix} 1 & 0 \end{bmatrix}$	$\begin{bmatrix} 1 & 0 \end{bmatrix}$
cat	$\begin{bmatrix} 1 & 1 & 0 \end{bmatrix}$	$\begin{bmatrix} 1 & 1 \end{bmatrix}$	$\begin{bmatrix} 1 & 0 \end{bmatrix}$
slept	$\begin{bmatrix} 1 & 1 & 1 \end{bmatrix}$	$\begin{bmatrix} 1 & 1 \end{bmatrix}$	$\begin{bmatrix} 1 & 1 \end{bmatrix}$

SHARED TOY PROJECTIONS (3×2)

$$\mathbf{W}_Q = \begin{bmatrix} 0 & 1 \\ 1 & 0 \\ 0 & 0 \end{bmatrix} \quad \mathbf{W}_K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{bmatrix} \quad \mathbf{W}_V = \begin{bmatrix} 1 & 0 \\ 0 & 0 \\ 0 & 1 \end{bmatrix}$$

The same three matrices are applied at every token position.

$$\mathbf{q}_3 = \mathbf{x}_3 \mathbf{W}_Q = \begin{bmatrix} 1 & 1 \end{bmatrix}$$

Only “slept” has a 1 in the second value coordinate.

At inference, only ⟨BOS⟩, The, cat are available when predicting “slept”.

Chosen embeddings and weights for illustration; positional encoding is omitted from this calculation.

An unmasked weighted sum can carry information from a token that is unavailable at inference.

Causal Masking

Compute the attention weights at “cat” and inspect the contribution from the future token “slept”.

QUERY AT POSITION 3; TRAINING TARGET: slept

$$\mathbf{q}_3 = \begin{bmatrix} 1 & 1 \end{bmatrix}, \quad d_k = 2$$

$$e_{3j} = \frac{\mathbf{q}_3 \mathbf{k}_j^T}{\sqrt{2}}$$

$$e_{34} = \frac{1 \cdot 1 + 1 \cdot 1}{\sqrt{2}} \approx 1.414$$

	⟨BOS⟩	The	cat	slept (future)
Scaled scores	0	0.707	1.414	1.414
Unmasked weights	0.089	0.180	0.365	0.365
Masked scores	0	0.707	1.414	$-\infty$
Masked weights	0.140	0.284	0.576	0

OUTPUT WITHOUT THE MASK

$$\mathbf{h}_3^{\text{unmasked}} \approx \begin{bmatrix} 0.911 & 0.365 \end{bmatrix}$$

$$\mathbf{h}_3 = \sum_{j=1}^4 \alpha_{3j} \mathbf{v}_j$$

Weights rounded to three decimals.

OUTPUT WITH THE MASK

$$\mathbf{h}_3^{\text{masked}} \approx \begin{bmatrix} 0.860 & 0 \end{bmatrix}$$

**The second coordinate falls from 0.365 to 0:
“slept” contributes nothing to the masked output.**

Apply the mask before softmax: future weights become zero, and the permitted weights are renormalized.